

Angular Training

The training should cover the below aspects –

1. How to structure Angular apps for scalability and maintainability.
2. Training sessions enabling the team to write clean, reactive and maintainable Angular code using RxJS and Signals.
3. Guidance to improve code quality, performance, and maintainability using proven principles.

Requirements:

1. Reactive Programming (RxJS & Signals)

Team needs a session on different programming paradigms like imperative, declarative, and reactive etc before the RxJS session.

Topics to Cover:

- **Reactive Programming with Angular**
 - What is reactive programming? Why Angular uses RxJS?
 - Push vs Pull architecture.
- **RxJS Fundamentals**
 - Observables, Subjects, BehaviorSubject, ReplaySubject.
 - Hot vs Cold Observables.
- **RxJS Operators**
 - Performance optimizations
 - Transformation
 - Filtering
 - Error handling
 - Retry mechanisms
- **Best Practices**
- **Signals in Angular (v16+)**
- **Hands-on session**

2. Scalable Architecture & Component Patterns

Topics to Cover:

- **Angular Architecture Overview**
 - Standalone components vs NgModules.
 - Feature-first folder structure.
- **Component Patterns**
 - Smart vs Presentational components.
 - SCAM (Single Component Angular Module) pattern.
- **State Management**
 - Component Store vs NgRx vs Signals-based store.
 - When to choose which.
- **Dependency Injection Best Practices**
 - inject() vs constructor DI.
- **Routing & Lazy Loading**
 - Route-level code splitting.
 - Guards (canMatch, canActivate).
- **Hands-on Session**
 - Creating standalone components with OnPush strategy.

3. Angular Best Practices, Performance, SOLID & Design Patterns

Topics to Cover:

- **Angular Best Practices**
- **Performance-Oriented Development**
- **SOLID Principles in Angular**
 - **Single Responsibility:** Components should do one thing.
 - **Open/Closed:** Extend without modifying.
 - **Liskov Substitution:** Interface segregation for services.
 - **Interface Segregation:** Avoid fat interfaces.
 - **Dependency Inversion:** Use DI properly.

- **Design Patterns for Angular**
 - **Facade Pattern** for state management.
 - **Adapter Pattern** for API responses.
 - **Observer Pattern** (RxJS core concept).
 - **Strategy Pattern** for dynamic behavior.
- **Anti-Patterns to Avoid**
- **Hands-on Session**
 - Implement Facade + Adapter pattern in a feature module.
 - Optimize a component for performance using OnPush & Signals.